

16-Agent Autonomous Engineering Workflow

RAM Disk Optimization for Multi-Agent AI Development

80,000

Lines of Code / Hour

14

Concurrent Agents

13

Repositories

22

LOC/sec (net)

Matthew Charles Busel

Tensorust | Oxidized Archive

March 17, 2026

github.com/Mattbusel

Contents

1. Overview — What This Is and Why It Matters
2. Architecture — The Three-Layer Agent Model
3. Infrastructure — RAM Disk Setup and Optimization
4. Session Workflow — Daily Operations
5. Performance Benchmarks — Self-Reported Agent Metrics
6. 10-Day Projection — Expected Output at Scale
7. Comparison — Industry Context
8. Scripts Reference — Complete Toolkit
9. Repository List — Target Codebases
10. Risks and Mitigations

1. Overview

This document describes a multi-agent software engineering workflow that runs 16 concurrent Claude Code agents across 13+ Rust and C++ repositories simultaneously. The agents autonomously identify improvements, implement changes, compile code, run tests, and commit to GitHub — with human oversight limited to architectural approval and judgment review.

The system hit a physical bottleneck: consumer NVMe SSDs cannot sustain the concurrent I/O pressure generated by 16 agents doing simultaneous Rust compilation and git operations. This was solved by moving the entire workspace to a 10GB RAM disk, eliminating disk I/O entirely and increasing per-agent throughput by 40-60%.

Key result: ~80,000 lines of tested, compiled code per hour across the fleet, with zero SSD writes during operation. The machine runs at 3% CPU, 2% disk, and is completely silent.

2. Architecture — The Three-Layer Agent Model

Layer 1: Directed Agents (4 agents)

Four agents receive specific tasks from the operator. These handle high-priority work where the operator has a clear architectural vision — feature implementations, major refactors, or critical bug fixes that require human-specified scope.

Example prompt: *"Pull from github.com/Mattbusel/math-sonify. I need audio and sound improvements — get the highest quality I can so it sounds like something that can turn into music."*

Layer 2: Autonomous Discovery Agents (12 agents)

Twelve agents receive a repository URL and a single instruction: explore the codebase, generate a list of improvements you would make, and wait for approval. The agents independently crawl source files, identify bugs, missing features, architectural gaps, performance issues, and code hygiene problems. They present a numbered task list. The operator reviews and approves.

This is **self-directed task generation** — the agents discover work rather than receiving it. The operator's role shifts from "write code" to "review judgment." The question is no longer "how do I implement this?" but "did the agent correctly identify what matters?"

Example prompt: *"Pull from this repo. I want a list of improvements I can do that aren't sound, UI, and bug related."*

Layer 3: Audit Agent (1 agent)

A dedicated audit agent runs at the end of each session. It inspects the git log, working tree, and commit history to produce a structured summary of everything that shipped — organized by area, with

LOC counts and file-level detail. This replaces manual code review for high-volume sessions where tracking 70+ commits across 13 repos would be impractical.

The operator's role: Taste, judgment, and approval. Reviewing agent-generated task lists, approving or rejecting proposals, and validating the audit summary. Not writing code.

3. Infrastructure — RAM Disk Setup and Optimization

The Problem: SSD Write Saturation

16 concurrent agents performing Rust compilation generate extreme disk I/O. Each cargo build produces thousands of intermediate files (.rlib, .rmeta, .o, .d) in target/ directories. Multiply by 13 repositories with overlapping builds, add concurrent git index operations, Windows Defender real-time scanning, and NTFS metadata journaling — the consumer NVMe's SLC write cache exhausts after approximately one hour. Write speeds cliff from ~5,000 MB/s to ~500 MB/s, and the session is effectively dead.

The Solution: Six-Phase Optimization

Phase	Optimization	Impact
1	10GB RAM disk via OSFMount at R:\	Eliminates all SSD writes. RAM: 10-20 GB/s, zero queue contention.
2	Git performance tuning (fsyncObjectFiles off, untrackedCache, preloadIndex, gc.auto 0)	Removes per-object fsync under concurrent git operations. ~10-15% throughput gain.
3	Windows Defender exclusions (all workspace paths, tool binaries, R:\ drive)	Eliminates real-time antivirus scanning overhead. ~20-30% throughput gain.
4	Cargo build optimization (CARGO_TARGET_DIR=R:\cargo-target, sparse registry protocol)	All Rust build artifacts go to RAM. Shared target dir deduplicates across repos. L
5	NTFS / Windows tuning (disable last-access time, 8.3 names, increased FS memory allocation)	Reduces NTFS metadata overhead. ~5-10% additional gain.
6	One-click launcher scripts (start-session.ps1, end-session.ps1)	Automates entire setup/teardown cycle. Mount, clone, optimize, dashboard, syn

Combined effect: Session duration increases from ~1 hour to 3-4+ hours. Per-agent task throughput increases 40-60%. The new bottleneck becomes the API usage window, not hardware.

System Resources During Operation

Resource	Before Optimization	After Optimization
CPU	3-5%	3%

Disk	80-100% (saturated)	2% (idle)
Memory	~45%	~68% (10GB RAM disk)
Noise	Fan spinning (SSD thermal)	Dead silent
Session limit	~1 hour	3-4+ hours (API-limited)

4. Session Workflow — Daily Operations

One-Time Setup (run once, survives reboots)

1. `.\configure-defender.ps1` — Run as Administrator. Adds all process and path exclusions.
2. `.\optimize-windows.ps1` — Run as Administrator. NTFS tweaks + High Performance power plan. Reboot after.
3. `.\optimize-cargo.ps1` — Normal user. Writes `~/.cargo/config.toml`, sets `CARGO_TARGET_DIR`.

Daily Session Flow

Step	Action	What Happens
1	Run <code>start-session.ps1</code>	Mounts 10GB RAM disk at <code>R:\</code> , clones or pulls all 13 repos, applies git optimizations, sets env vars, prints status dashboard.
2	Open 16 Claude Code terminals	4 terminals get directed task prompts. 12 terminals get autonomous discovery prompts.
3	Approve agent task lists	Review each autonomous agent's proposed improvements. Approve, reject, or modify scope.
4	Monitor (optional)	Agents execute autonomously. Watch YouTube, work on other tasks, or review commits as they land.
5	Run audit agent	Dedicated agent produces structured summary of all changes, organized by area with LOC counts.
6	Run <code>end-session.ps1</code>	Syncs all repos to GitHub via git push. Optionally backs up to SSD. Unmounts RAM disk.

Agent Prompt Templates

Directed agent:

```
https://github.com/Mattbusel/[repo] pull from here I need [specific task description] – I need the highest quality I can get
```

Autonomous discovery agent:

```
https://github.com/Mattbusel/[repo] pull from here I want a list of improvements I can do that aren't [excluded categories]. Wait for my approval before implementing.
```

Audit agent:

[https://github.com/Mattbusel/\[repo\]](https://github.com/Mattbusel/[repo]) pull from here I need you to run an audit – tell me what has been done, what changed, LOC counts, and a full summary of all commits.

5. Performance Benchmarks — Self-Reported Agent Metrics

Each agent was instructed to self-report its performance metrics at the end of its session. These are real measurements from the March 17, 2026 session, post-RAM disk optimization.

Agent	Repository	Net LOC	Wall-Clock	LOC/sec	LOC/min
1	Every-Other-Token	3,126	~25 min	2.1	~125
2	Every-Other-Token	3,126	~30 min	1.74	~104
3	Every-Other-Token	1,051	~8 min	2.2	~131
4	HelixRouter	2,565	~24 min	1.78	~107
5	agent-runtime	2,930	~12 min	4.1	~244
6	HelixRouter	845	~47 min	0.30	~18
7	tokio-prompt-orchestrator	1,098	~25 min	0.73	~44

Metric	Value
Average LOC/sec per agent	~1.56
Fleet LOC/sec (14 agents)	~21.8
Fleet LOC/min	~1,344
Fleet LOC/hour	~80,000
Average improvements per agent	~16
Fleet improvements per hour	~224
Tests generated per session	171+ (all passing)
API usage per session	~35-40% of 5-hour rolling window

Note on Agent 5 (agent-runtime): This agent hit 4.1 LOC/sec — the highest recorded — because it performed a deep architectural rewrite: async tool handlers, vector-based semantic search, LLM provider integrations, state persistence, Dijkstra's algorithm, Brandes' betweenness centrality, typestate builder validation, lock poisoning recovery across 30 call sites, and distributed circuit breakers. All with 171 passing tests in 12 minutes.

6. 10-Day Projection — Expected Output at Scale

Based on the March 17 benchmark data, the following projections assume 14 agents at ~80,000 LOC/hour with ~35-40% API usage per session on a 5-hour rolling window.

Pace	Sessions/Day	Lines/Day	Lines/10 Days	Improvements/10 Day
Conservative	3	~185,000	~1,850,000	~27,000
Moderate	4	~246,000	~2,460,000	~36,000
Aggressive	5	~308,000	~3,080,000	~45,000

Expected Qualitative Outcomes by Phase

Days	Expected Behavior
1-3	Explosive growth. Agents find the most obvious improvements across every repo — bugs, missing tests, architectural gaps, performance wins. Bulk of LOC lands here.
4-6	Refinement phase. Agents shift from feature addition to optimization, edge case handling, and deeper test coverage. LOC may drop but quality per line increases.
7-10	Diminishing returns per repo. Smaller, more surgical improvements. This is the most valuable phase — the long tail of correctness that separates projects from products.

Expected Repository State After 10 Days

Repository	Expected State
tokio-prompt-orchestrator	Production-grade. Full observability, distributed tracing, chaos testing, battle-hardened error handling. Enterprise-ready.
math-sonify	v5-v6. Every major dynamical system implemented. Professional DSP chain. Polished UI. A real product, not a project.
Every-Other-Token	Full research platform. Web UI, CLI, experiment replay, A/B testing infrastructure, statistical significance testing.
LLMTokenStreamQuantEngine	Institutional-grade C++. Every atomic, lock, and SIMD path validated. Production-correct FIX protocol handling.
agent-runtime	Standalone framework. Persistence backends, provider integrations, graph analytics, circuit breakers. Publishable as a library.

Long-tail repos (8)

All hardened, documented, tested, CI-passing. Every repo in the portfolio looks actively maintained by a dedicated team.

7. Comparison — Industry Context

To contextualize the output volume, the following comparisons use publicly available estimates of engineering productivity and codebase sizes.

Benchmark	Metric	This Workflow
Senior engineer (daily)	100-200 LOC/day	Equivalent output every 8-15 seconds
Google engineer (annual)	~66,000 LOC/year	Equivalent output in ~1 hour
8-10 person startup (weekly)	5,000-10,000 LOC/week	Equivalent output in 4-8 minutes
Linux kernel (total)	~28M LOC over 30+ years	10-day projection: 7-10% of total volume
Windows OS (total)	~50-60M LOC over decades	10-day projection: ~4-5% of total volume

Ecosystem Comparison: Multi-Agent Approaches

Approach	Typical Agent Count	Coordination	I/O Optimization
Claude Code subagents	2-4	Within single session	None
Gas Town (Yegge)	3 (across 3 Max agents)	Kubernetes-like orchestration	None
Multiclaude	3-6	Primary + subagent pattern	None
Claude Squad	3-8	Git worktree isolation	None
Agent Teams (official)	3-5 (research preview)	Git-based task claiming	None
This workflow	16 concurrent	Autonomous discovery + human approval gate + audit agent	10GB RAM disk, git tuning, Defender exclusions, CARGO_TARGET_DIR redirect

Key differentiator: No other published multi-agent workflow includes I/O-level infrastructure optimization. The entire ecosystem focuses on orchestration software. This workflow is the first known instance of engineering the hardware layer to sustain agent concurrency beyond consumer SSD limits.

8. Scripts Reference — Complete Toolkit

All scripts live in `scripts/agent-perf/` within the repository.

Script	Admin?	Purpose
start-session.ps1	No	Daily launcher. Checks/creates RAM disk, pulls repos, sets env vars, prints status dashboard with all 6 subsystems.
end-session.ps1	No	Daily closer. Syncs all repos to GitHub, optionally backs up to SSD, unmounts RAM disk.
setup-ramdisk.ps1	No*	Creates 10GB R:\ via OSFMount, clones all 13 repos, sets TMPDIR/TMP/TEMP.
sync-ramdisk.ps1	No	Runs git push --all for every repo + optional SSD backup.
teardown-ramdisk.ps1	No	Runs sync, then unmounts RAM disk with force-detach fallback.
optimize-git.ps1	No	Applies 13 performance settings to every repo. fsyncObjectFiles=false is the largest single win.
configure-defender.ps1	Yes	Excludes R:\, workspaces, .cargo, .rustup, all target\ dirs, and tool binaries from real-time scanning.
revert-defender.ps1	Yes	Removes all added Defender exclusions.
optimize-cargo.ps1	No	Writes ~/.cargo/config.toml. Sets CARGO_TARGET_DIR=R:\cargo-target. Detects sccache and lld.
optimize-windows.ps1	Yes	NTFS tweaks (last-access time, 8.3 names, memory allocation) + High Performance power plan.
README.md	—	Full documentation with expected performance numbers and risk table.

9. Repository List — Target Codebases

Repository	Language	Description
tokio-prompt-orchestrator	Rust	Multi-stage LLM pipeline with backpressure, dedup, and adaptive control
math-sonify	Rust	Dynamical systems sonification engine — chaos-to-audio synthesis
Every-Other-Token	Rust	Real-time LLM token stream interceptor for interpretability research
agent-runtime	Rust	Agent framework: ReAct loops, memory, graph analytics, circuit breakers
HelixRouter	Rust	Adaptive async compute router with neural routing and epsilon decay
fin-stream	Rust	Financial data streaming with circuit breakers and bar emission
fin-primitives	Rust	Financial primitives: RSI, VWAP, spread guards, order book types
geodesic-wallpaper	Rust	Geodesic visualization and wallpaper generation
llm-cost-dashboard	Rust	LLM API cost tracking and analytics dashboard
Reddit-Options-Trader-ROT	Rust/Python	Financial intelligence platform — Reddit sentiment to options signals
FinRL_DeepSeek_Crypto_Trading	Python	Deep reinforcement learning for crypto trading
LLMTokenStreamQuantEngine	C++	Real-time quant engine consuming LLM token streams via FIX 4.2 OMS
Special-Relativity-in-Financial-Modeling	Rust	Lorentz-covariant price dynamics on OHLCV data (SRFM)

10. Risks and Mitigations

Risk	Severity	Mitigation
RAM disk data loss on crash or power failure	Medium	All committed work pushes to GitHub. Uncommitted work lost. end-session.ps1 syncs before teardown. Optional SSD backup path.
Agent introduces bugs	Low-Medium	Rust compiler catches most issues at compile time. CI pipelines run on push. Audit agent reviews all changes. Property-based and chaos tests catch edge cases.
Agent conflicts (overlapping changes)	Low	Each agent works on a separate repo or separate files within a repo. Git merge conflicts are rare and agents resolve them autonomously.
API usage limit hit mid-session	Low	35-40% per session leaves headroom for 2-3 sessions per window. Agents gracefully stop when rate-limited.
RAM disk fills up (>10GB)	Low	CARGO_TARGET_DIR shared across repos reduces duplication. 10GB provides headroom for 16 agents' peak concurrent builds. Oscillation pattern (4-9GB) confirms artifacts are cleaned up.
SSD wear from RAM disk syncs	Negligible	Sync only happens at session end (1x per session). Daily write volume to SSD drops from hundreds of GB to <1GB.

This document and the associated scripts are open source. If you replicate this workflow or improve upon it, reach out — I'd like to hear about it.

Matthew Charles Busel — Tensorust — github.com/Mattbusel — LinkedIn: [Matthew Charles Busel](#)